PLAN DE PRUEBAS

Plataforma Tu Evento

Equipo: CapySoft
Programa: Análisis y Desarrollo de Software (ADSO), SENA, ficha 3145556
Instructora: Karol Daniela Correa
Líder del equipo: Cristofer David Lozano Contreras
Versión: 1.0
Fecha: 9 de octubre de 2026

1. Introducción y objetivo

Tu Evento es una plataforma de CapySoft para crear, publicar y operar eventos, con venta de boletas por silla, pagos, wallet de crédito, notificaciones y validación de ingreso por QR. Este documento define cómo se verificará que el sistema cumple lo esperado antes de entregarlo.

Objetivo general: comprobar que las funcionalidades del backend (Spring Boot), la web (React) y la app móvil (React Native/Expo) se comportan según las reglas de negocio, que los permisos por rol se respetan y que los flujos críticos de compra, pago y reembolso son confiables.

Objetivos específicos

- Validar los flujos de registro, activación, login y recuperación de acceso.
- Validar el ciclo de vida del evento, desde DRAFT hasta PUBLISHED, CANCELLED o COMPLETED.
- Validar la reserva de sillas, la orden, el pago y el webhook, incluida la concurrencia.
- Validar los reembolsos y el uso del wallet (solo wallet, mixto y por pasarela).
- Verificar la autorización por rol (USER, ORGANIZER, ADMIN) y detectar huecos de seguridad conocidos.

2. Alcance

Dentro del alcance

- Autenticación (32 casos)
- Usuarios (9 casos)
- Administración (6 casos)
- Perfil (5 casos)
- Eventos (21 casos)
- Media y layout (7 casos)
- Ratings y comentarios (6 casos)
- Categorías y sitios (7 casos)
- Secciones y sillas (9 casos)
- Órdenes y tickets (12 casos)
- Pagos (16 casos)
- Wallet (5 casos)
- Notificaciones (4 casos)
- Temas e idiomas (6 casos)
- Seguridad y almacenamiento (6 casos)

Fuera del alcance

- Pruebas de carga y rendimiento extremo, salvo la verificación básica de concurrencia de sillas.
- Pasarela de pagos real (Wompi): hoy el pago se simula con la pasarela falsa propia del equipo.
- Notificaciones push nativas (FCM/APNs): no están implementadas.

- Lector de QR en los clientes: no existe interfaz, el check-in se prueba por API.
- Pipeline DevOps (Jenkins, Sonar, Kubernetes) y la infraestructura de Redis.

3. Tipos de prueba y estrategia

Tipo	Qué se verifica	Herramienta
Funcional	Cada endpoint y pantalla con datos válidos e inválidos	Postman o Swagger (/swagger-ui) y navegador
Límites	Valores extremos (aforo 1 y 100000, textos de 255 y 256 caracteres)	Postman
Integración	Backend con pasarela fake, MinIO, SMTP Gmail, LibreTranslate	Flujo completo manual
Seguridad	Roles, JWT, firma HMAC del webhook, redirecciones OAuth	Postman con distintos tokens
Concurrencia	Dos usuarios sobre la misma silla	Dos sesiones o dos peticiones simultáneas
Aceptación	Flujos de punta a punta por rol en web y móvil	Navegador y Expo Go

Enfoque: las pruebas son manuales sobre la API y la interfaz. Los casos negativos se mapean al campo code de la respuesta de error. Cada caso se ejecuta con el procedimiento descrito en el archivo Casos_de_Prueba_TuEvento.xlsx, que es el documento complementario de este plan.

4. Entorno de pruebas

Componente	Configuración
Backend	Spring Boot, monolito con prefijo /api/v1, perfil de desarrollo
Base de datos	PostgreSQL 17 con Liquibase
Almacenamiento	MinIO (API S3)
Pasarela de pagos	fake-payment-gateway en el puerto 4001, con webhook firmado con HMAC (payment.gateway: fake)
Correo	SMTP Gmail para activación, recuperación y notificaciones
Traducción	LibreTranslate en Docker
Moderación de imágenes	Servicio NSFW local y Sightengine
Web	React + Vite en navegador de escritorio actual
Móvil	Expo / React Native con Expo Go
Usuarios de prueba	Un USER, un ORGANIZER y un ADMIN con correos @gmail.com reales para recibir códigos

Nota: el endpoint GET /auth/dev/activate-code/{email} sirve para obtener el código de activación sin abrir el correo, solo en el entorno de pruebas.

5. Criterios de entrada y salida

Criterios de entrada

- Backend, web y móvil desplegados en el entorno de pruebas y sin errores de arranque.
- Pasarela fake, MinIO, PostgreSQL y SMTP disponibles.
- Usuarios de prueba creados para cada rol y al menos un sitio y una categoría activos.
- Casos de prueba revisados por el equipo.

Criterios de salida

- 100 % de los casos de prioridad Alta ejecutados.
- Al menos 95 % de los casos de prioridad Alta aprobados.
- Ningún defecto abierto de severidad crítica en pagos, órdenes o seguridad.
- Los defectos restantes registrados con su prioridad y decisión (corregir o aceptar).

Criterios de suspensión

Se detienen las pruebas si el backend no inicia, si la base de datos no responde o si la pasarela fake no está disponible, ya que bloquean los flujos de compra.

6. Roles y responsabilidades

Rol	Responsabilidad
Líder del equipo	Aprobar el plan, priorizar defectos y decidir el cierre
Diseñador de pruebas	Redactar y mantener los casos de prueba
Ejecutor	Ejecutar los casos, registrar resultado obtenido y evidencias
Desarrollo (backend/web/móvil)	Corregir defectos y confirmar la causa
Instructora	Revisión académica del documento

Como CapySoft es un equipo de dos personas, es habitual que una misma persona cumpla varios roles. Se recomienda que quien ejecuta un caso no sea quien escribió el código que prueba.

7. Cronograma

Fase	Actividad	Duración estimada	Responsable
1	Revisión del plan y los casos	1 día	Equipo
2	Pruebas de autenticación, usuarios y perfil	2 días	Ejecutor
3	Pruebas de eventos, layout y sillas	2 días	Ejecutor
4	Pruebas de órdenes, pagos, wallet y reembolsos	3 días	Ejecutor
5	Pruebas de seguridad, notificaciones, temas e idiomas	1 día	Ejecutor
6	Corrección de defectos y regresión	2 días	Desarrollo

Fase	Actividad	Duración estimada	Responsable
7	Informe final de resultados	1 día	Líder

Las duraciones son estimadas y deben ajustarse a las fechas de entrega del SENA.

8. Resumen de casos de prueba

El archivo Casos_de_Prueba_TuEvento.xlsx contiene 151 casos distribuidos así:

Módulo	Casos
Autenticación	32
Usuarios	9
Administración	6
Perfil	5
Eventos	21
Media y layout	7
Ratings y comentarios	6
Categorías y sitios	7
Secciones y sillas	9
Órdenes y tickets	12
Pagos	16
Wallet	5
Notificaciones	4
Temas e idiomas	6
Seguridad y almacenamiento	6
Total	151

Cada caso incluye identificador, referencia al requisito o sección del inventario técnico (trazabilidad), tipo, prioridad, precondiciones, pasos, datos de entrada y resultado esperado. Las columnas de resultado obtenido, estado y evidencia se llenan al ejecutar. La referencia de cada caso apunta a una sección del inventario técnico (Anexo A).

9. Riesgos

Riesgo	Impacto	Mitigación
Pasarela fake distinta de la real (Wompi)	Alto	Documentar la diferencia y repetir pruebas de pago al integrar la pasarela real
Dependencia de servicios externos (SMTP, Google OAuth, Sightengine)	Medio	Probar con cuentas de prueba y registrar fallos de red como bloqueados

Riesgo	Impacto	Mitigación
Casos de concurrencia difíciles de reproducir	Medio	Usar dos sesiones y repetir varias veces
Móvil con menos funciones que la web	Medio	Limitar los casos móviles a los flujos disponibles
Huecos de seguridad conocidos sin corregir	Alto	Incluir casos específicos y reportarlos como defectos
Bloqueo de cuenta de solo 1 minuto en desarrollo	Bajo	Considerarlo al probar el límite de intentos

10. Hallazgos conocidos antes de ejecutar

El análisis del código identificó comportamientos que probablemente fallarán en los casos de seguridad. Se incluyen como casos para confirmarlos y registrarlos como defectos:

- GET /tickets/user/{userId} no verifica que el solicitante sea el mismo usuario.
- POST /orders/{id}/cancel no comprueba el dueño de la orden.
- POST /tickets/{id}/checkin puede ejecutarlo cualquier usuario autenticado, no existe rol de staff.
- Las rutas de storage están como públicas en SecurityConfig.
- El endpoint /auth/dev/activate-code existe y debe deshabilitarse en producción.
- Actuator es público.
- El listado global de jobs de traducción pendientes devuelve una lista vacía.

11. Gestión de defectos

Cada defecto se registra con: identificador, caso de prueba asociado, descripción, pasos para reproducirlo, resultado esperado y obtenido, evidencia (captura o respuesta de la API), severidad y estado.

Severidad	Criterio	Ejemplo
Crítica	Impide un flujo principal o compromete datos o dinero	Una orden pagada deja las sillas libres
Alta	Falla una función importante con alternativa difícil	Un USER puede hacer check-in
Media	Falla parcial con alternativa	Mensaje de error incorrecto
Baja	Defecto cosmético o menor	Texto desalineado

Estados de un defecto: Nuevo, En corrección, Corregido, Verificado y Cerrado. Todo defecto corregido exige repetir el caso que lo detectó y los casos relacionados.

12. Referencias

CapySoft. (2026). Inventario técnico de Tu Evento para pruebas [Documento interno]. Se incluye completo como Anexo A de este documento.

CapySoft. (2026). Casos de prueba de Tu Evento (Casos_de_Prueba_TuEvento.xlsx) [Documento interno].

Anexo A. Inventario técnico de Tu Evento para pruebas

Levantamiento del estado actual del sistema (backend, web y móvil), elaborado con apoyo de un asistente de IA a partir del análisis del código. Describe lo que el sistema hace hoy, no los requisitos originales, y debe contrastarse con el código y con los requisitos antes de darse por definitivo. Fecha del levantamiento: octubre de 2026.

Documento de pruebas basado en el código actual de backend, web y móvil. Las respuestas de la API van, en la mayoría de casos, en el sobre `{ success, message, code, data }`. Los pagos (crear y consultar) y algunos de traducción/admin no usan ese sobre.

1. Qué es Tu Evento y módulos principales

Plataforma de CapySoft para crear, publicar, vender boletas y operar eventos (layout, sillas, pagos, QR). El backend Spring Boot es el núcleo. Hay frontend web (Vite/React) y app móvil (Expo/React Native). El README aún habla de Wompi y API Gateway; en el código de desarrollo el pago corre contra una **pasarela falsa** (`payment.gateway: fake`).

Backend (módulos)

Seguridad, perfil, evento (media, layout, ratings/comentarios), categoría, geolocalización, sección, asiento, ticket/orden, pago, wallet, notificación, tema, idioma/traducción, almacenamiento.

Web

Landing, catálogo y detalle de eventos, wizard de creación, editor de layout, gestión de eventos, checkout y confirmación de pago, wallet, perfil, notificaciones, petición de organizador, paneles de admin (peticiones y eventos). El diseño de venue es exclusivo de web.

Móvil

Landing, registro/login/activación, recuperación de contraseña, listado y detalle, checkout y pago pendiente, perfil/temas, y pantallas de admin para peticiones de organizador. No hay wizard de creación ni editor de layout. No hay pantalla de escáner QR de ingreso.

2. Convención de errores HTTP (aplicable a casi todos los endpoints)

Código	Cuándo
200 / 201	Éxito (201 en altas: registro, evento, orden, pago, perfil, sitio)
400	Validación Bean Validation, JSON mal formado, `BusinessException`, `IllegalStateException`
401	Sin JWT, JWT inválido/expirado, firma HMAC de webhook inválida, `AuthenticationException`
403	Autenticado sin rol/autoridad (`Access denied`)
404	Recurso no encontrado (`NotFoundException`)
405	Método HTTP no soportado
409	Integridad (`ConstraintViolationException`), sitio/perfil duplicado, transición inválida de pago u orden
422	Política de imagen (moderation) o saldo de wallet insuficiente
500	Error no controlado (`An unexpected error occurred`)

En 400 de negocio el campo `code` es el identificador para mapear pruebas (ej. `EVENT_START_DATE_IN_PAST`). JWT expirado: el cliente web intenta refresh; 403 no dispara refresh.

3. Endpoints y pantallas (método, ruta, parámetros, respuestas, errores)

Prefijo de API: `/api/v1`, salvo traducción admin (`/api/admin/translations`). Autenticación: header `Authorization: Bearer <accessToken>` salvo rutas públicas.

3.1 Autenticación (/api/v1/auth) — público

Método	Ruta	Parámetros	Éxito	Errores típicos
POST	`/register`	Body: `email`, `password`, `fullName`	201. Cuenta creada; hay que activar por correo	400 `INVALID_EMAIL` (solo Gmail), `INVALID_PASSWORD`, `INVALID_NAME`, `EMAIL_ALREADY_EXISTS`, `EMAIL_NOT_ACTIVATED`
POST	`/activate`	`email`, `activationCode`	200	400 `ACCOUNT_ALREADY_ACTIVATED`, `INVALID_ACTIVATION_CODE`, `ACTIVATION_CODE_EXPIRED`
POST	`/resend-activation`	`email`	200	400 `EMAIL_ALREADY_ACTIVATED`
POST	`/login`	`email`, `password`	200: tokens, `userId`, `alias`, `role`	404 `USER_NOT_FOUND`; 400 `ACCOUNT_NOT_ACTIVATED`, `ACCOUNT_BLOCKED`, `ACCOUNT_DEACTIVATED`, `ACCOUNT_DELETED`, `INVALID_CREDENTIALS`, `ACCOUNT_LOCKED`
POST	`/logout`	`accessToken`	200	400 `UNAUTHORIZED_LOGOUT`
POST	`/refresh`	`refreshToken`	200 nuevos tokens	400 `REFRESH_TOKEN_REVOKED`, `REFRESH_TOKEN_EXPIRED`, `ACCOUNT_NOT_ACTIVE`
POST	`/recover-password`	`email`	200 (mensaje genérico; no enumerar usuarios)	—
POST	`/reset-password`	código + nuevas contraseñas	200	400 `PASSWORD_MISMATCH`, `INVALID_RECOVERY_CODE`, `RECOVERY_CODE_USED`, `RECOVERY_CODE_EXPIRED`
GET	`/oauth/{provider}`	Query: `state`, `frontend_redirect_uri`	302 al proveedor	400 si el provider no existe
GET	`/oauth/{provider}/callback`	`code`, `state`; `response_format=json` en móvil	302 al frontend (web) o 200 JSON (móvil)	Fallos OAuth
POST	`/google`	ID token GSI	200 login	400 `GOOGLE_TOKEN_INVALID`, `GOOGLE_EMAIL_NOT_VERIFIED`
POST	`/reactivate-request`	`email`	200 siempre (anti-enumeración)	—
POST	`/reactivate-confirm`	token de reactivación	200	400 token inválido/expirado
GET	`/dev/activate-code/{email}`	email	200 código (solo pruebas)	404 `EMAIL_NOT_FOUND`; 400 `NO_ACTIVATION_CODE`

Pantallas web: `/login` (incluye registro, forgot password embebido, OAuth Google/Facebook), `/verification`, `/reactivate-account`.

Pantallas móvil: Landing, Login, Register, Terms, ForgotPassword, ResetPassword, Activate.

3.2 Usuarios autenticados (/api/v1/users)

Método	Ruta	Parámetros	Notas / errores
POST	`/change-password`	actual, nueva, confirmación	400 `PASSWORD_MISMATCH`, `INVALID_CURRENT_PASSWORD`, `PASSWORD_RECENTLY_USED`

Método	Ruta	Parámetros	Notas / errores
POST	`/oauth/link`	proveedor + código	400 `OAUTH_ALREADY_LINKED`, `OAUTH_PROVIDER_TAKEN`
POST	`/organizer-request`	multipart: `document`, `documentType` opcional	400 `ALREADY_ORGANIZER`, `PETITION_ALREADY_PENDING`, `DOCUMENT_READ_ERROR`
GET	`/organizer-petition`	—	200 o 404 sin petición
DELETE	`/me`	contraseña	400 `INVALID_PASSWORD`, `ACCOUNT_ALREADY_INACTIVE`, `ACCOUNT_DELETED`

Web: `/organizer-petition-form`, cambio de contraseña y desactivar cuenta desde perfil. **Móvil:** Settings, EditProfile; no hay flujo de petición de organizador para el usuario (sí pantallas admin).

3.3 Administración (/api/v1/admin) — rol ADMIN

Método	Ruta	Parámetros	Errores
GET	`/organizer-requests`	—	401/403
PUT	`/organizer-requests/{id}/approve`	—	400 `PETITION_NOT_PENDING`; 404
PUT	`/organizer-requests/{id}/reject`	—	ídem
GET	`/organizer-requests/{id}/document`	—	404 `PETITION_NOT_FOUND`, `FILE_NOT_FOUND`
GET	`/events`	query `status` opcional	—
PATCH	`/events/{eventId}/status`	`newStatus`, `reason` (obligatorio al rechazar)	ver transiciones admin

Web: `/admin-panel`, `/admin-events`. **Móvil:** OrganizerRequests, OrganizerRequestDetail.

3.4 Perfil (/api/v1/profiles)

GET por id o por userId es público según SecurityConfig. POST/PUT requieren autenticación.

Método	Ruta	Body	Errores
POST	`/`	`userId`, `fullName` (máx. 100), `storedFileId` opcional	201; 409 perfil ya existe; 400 nombre/bio
PUT	`/{profileId}`	`cityId`, `storedFileId`, `fullName` (máx. 30), `bio` (máx. 255)	400 `BIO_INAPPROPRIATE_LANGUAGE`, `INVALID_NAME`
GET	`/{profileId}`	—	404
GET	`/user/{userId}`	—	404

Web: `/profile`, `/complete-profile`. **Móvil:** Perfil, EditProfile.

3.5 Eventos (/api/v1/events)

Públicos (GET): `/{id}`, `/public`, `/public/city/{cityId}`, `/public/category/{categoryId}`, `/public/date-range?from&to` (ISO date), `/status/{status}`.

Organizador: POST crear, PUT/DELETE/`PATCH .../status`. GET `/user/{userId}` autenticado.

Body crear: `eventName`, `description`, `siteId`, `startDate`, `finishDate`, `isPublic` (default true), `availableSeats` (1–100000), `categoryId`.

Errores de negocio: `EVENT_DATES_REQUIRED`, `EVENT_START_DATE_IN_PAST`, `EVENT_START_DATE_TOO_FAR` (más de 2 años), `EVENT_END_BEFORE_START`, `EVENT_ALREADY_EXISTS`

(mismo nombre+fecha+sitio), CATEGORY_NOT_FOUND_OR_INACTIVE, SEATS_EXCEED_CAPACITY, EVENT_ACCESS_DENIED, EVENT_DELETE_NOT_ALLOWED, EVENT_INVALID_STATUS_TRANSITION, EVENT_PUBLISH_MEDIA_COUNT_INVALID (3–9 imágenes), EVENT_SECTIONS_REQUIRED, EVENT_COMPLETE_TOO_EARLY, INVALID_DATE_RANGE.

Web: / landing, /events, /events/:eventId, /events/create, /events/manage.

Móvil: Inicio, Eventos, EventDetail.

3.6 Media, layout, ratings, comentarios

Método	Ruta	Quién	Notas
POST	/events/{eventId}/media	ORGANIZER/ADMIN	multipart; 400 EVENT_UPDATE_NOT_ALLOWED, EVENT_MEDIA_READ_ERROR; 422 política de imagen
GET	/events/{eventId}/media	público	—
GET/PUT	/events/{eventId}/layout	GET público; PUT autenticado	PUT: dueño; 400 EVENT_LAYOUT_EDIT_NOT_ALLOWED, SEAT_REGENERATION_CONFLICT, LAYOUT_PARSING_ERROR
POST	/events/{eventId}/ratings	autenticado	comentario obligatorio máx. 500; rating 1–5 opcional
GET	/events/{eventId}/ratings	público	—
POST/GET	/ratings/{ratingId}/replies	POST autenticado; GET público	—

Ratings: solo eventos PUBLISHED o COMPLETED y públicos. Un rating numérico por usuario (el resto son comentarios). El organizador no deja nota. Antispam: 1 comentario cada 10 s. Códigos: EVENT_RATING_NOT_ALLOWED, COMMENT_RATE_LIMITED, COMMENT_BLANK, COMMENT_TOO_LONG.

Web: /events/:eventId/layout (ORGANIZER/ADMIN).

3.7 Categorías

GET /categories, /active, /root, /{id}, /{id}/subcategories público.

POST/PUT/activate/deactivate/DELETE: ADMIN.

/category-events: GET público; POST/DELETE ORGANIZER o ADMIN.

Error: CATEGORY_NAME_ALREADY_EXISTS.

3.8 Geolocalización (/api/v1/geolocation) — GET públicos

Departamentos, ciudades, sitios por ciudad/departamento, sitio por id, cercanos (latitude, longitude, radiusKm).

POST/PUT sitios: ADMIN u ORGANIZER. Crear: cityId, name (150), address (200), capacity > 0, lat [-90,90], lon [-180,180]. Error 409 sitio duplicado.

3.9 Secciones y sillas

Método	Ruta	Auth	Body / notas
GET	/section-types	público	—
POST	/section-types	autenticado (admin en práctica)	nombre único; SECTION_TYPE_ALREADY_EXISTS
GET	/event-sections/event/{eventId}	público	—

Método	Ruta	Auth	Body / notas
POST/PUT/DELETE	`/event-sections`	ORGANIZER/ADMIN	capacidad ≥ 1, precio ≥ 0; `EVENT_SECTION_ALREADY_EXISTS`
GET	`/seat-blocks/section/{id}`	público	—
POST/DELETE	`/seat-blocks`	ORGANIZER/ADMIN	`SEAT_BLOCK_ALREADY_EXISTS`
GET	`/seats/block/{id}`, `/seats/section/{id}`, `/seats/{id}/log`	público	—
POST/PATCH/DELETE	`/seats`	ORGANIZER/ADMIN	`SEAT_CODE_ALREADY_EXISTS`, `SEAT_STATUS_UNCHANGED`
POST	`/seats/{seatId}/reserve`	autenticado	10 min TTL; `SEAT_NOT_AVAILABLE`
POST	`/seats/{seatId}/release`	autenticado	dueño o ADMIN; `SEAT_NOT_RESERVED`, `UNAUTHORIZED_RELEASE`

Estados de silla: AVAILABLE, RESERVED, SOLD, COURTESY. Tipos: regular y cortesía. Los cambios de silla se publican por WebSocket `/topic/events/{eventId}/seats`. Handshake WS en `/ws/**` con JWT.

3.10 Órdenes y tickets

Método	Ruta	Auth	Body	Errores
POST	`/orders`	autenticado	`eventId`, `seatIds` (mín. 1)	silla no reservada por el usuario, reserva vencida, silla inexistente → 400
GET	`/orders/{orderId}`	autenticado	—	404
GET	`/orders/{orderId}/tickets`	autenticado	—	404
POST	`/orders/{orderId}/cancel`	autenticado	—	409 si no está en DRAFT o PAYMENT_PENDING
GET	`/tickets/{ticketId}`	autenticado	—	404
GET	`/tickets/user/{userId}`	autenticado	**no verifica que sea el mismo usuario**	—
POST	`/tickets/{ticketId}/checkin`	autenticado	`validatedBy`	400 si no PAID, expirado o ya usado

No hay `@PreAuthorize` de ORGANIZER en check-in: cualquier usuario autenticado que conozca el id puede intentar validar. No hay UI de escáner.

3.11 Pagos (/api/v1/payments) y webhook

Método	Ruta	Auth	Body	Respuesta
POST	`/payments`	autenticado	`orderId`, `paymentMethod` opcional (default QR), `applyWalletCredit`	201 **sin** sobre ApiResponse; 400/422 si falla pasarela o saldo
GET	`/payments/{id}`	autenticado	—	200 PaymentResponse
POST	`/payments/{id}/refund`	ADMIN u ORGANIZER	`reason` (obligatorio, máx. 500)	200; solo si el pago está APPROVED
POST	`/webhooks/payment`	público + `X-Signature` HMAC	JSON crudo	200; 401 firma inválida; 500 error de proceso

Métodos de pago: CARD, PSE, NEQUI, CASH, QR.

Web: `/checkout`, `/checkout/pending/:paymentId`, `/checkout/confirmed/:paymentId`.

Móvil: Checkout, PaymentPending.

3.12 Wallet (/api/v1/wallet)

GET `/me` y `/me/transactions`: dueño. 404 si no hay wallet.

POST `/credit` y `/adjust`: ADMIN. Ajuste positivo suma, negativo resta; no deja balance negativo. 422 `InsufficientWalletBalance`.

Web: `/wallet`. Móvil: no hay pantalla de wallet.

3.13 Notificaciones (/api/v1/notifications)

GET `/me?page&size&unreadOnly` (page 0, size 20). GET `/me/unread-count`. PATCH `/{notificationUserId}/read`. PATCH `/me/read-all`. Solo el usuario autenticado.

Web: `/notifications` y campana. Móvil: no hay bandeja dedicada.

3.14 Temas (/api/v1/themes)

GET lista: público. POST activate, GET my-active, PUT customize, DELETE customize/{property}, GET log: autenticado. Temas: light, dark, purple, blue.

3.15 Idiomas

GET `/languages` y `/{id}`: público (activos). Admin: listar todos, crear, actualizar, activar/desactivar, set-default. Errores: `LANGUAGE_CODE_EXISTS`, `CANNOT_DEACTIVATE_DEFAULT_LANGUAGE`, `LANGUAGE_ALREADY_ACTIVE/INACTIVE`, `LANGUAGE_NOT_ACTIVE`, `LANGUAGE_ALREADY_DEFAULT`.

Traducción: `/api/admin/translations/request` (entityType, entityId, sourceLanguageCode), jobs por entidad, jobs pending (el listado global **devuelve lista vacía**: TODO en repositorio). Proveedor: LibreTranslate.

3.16 Storage (/api/v1/storage)

Marcado **público** en SecurityConfig (el upload internamente espera usuario). POST `/upload` multipart. GET archivo, por owner, URL. DELETE. Errores: `FILE_CATEGORY_NOT_FOUND`, `INVALID_FILE_EXTENSION`, `FILE_TOO_LARGE`, `UPLOAD_FAILED`, `S3_*`, 422 imagen inapropiada. Multipart global: 10 MB archivo / 12 MB request.

3.17 Otras pantallas web

`/nosotros`, `/terminos-de-uso`, `/privacidad`. Forgot password no es ruta propia. Existe `ChangePassword.jsx` y un demo de layout, no expuestos como rutas principales.

4. Reglas de negocio y validaciones

Cuenta

Correo solo `@gmail.com`. Contraseña: mínimo 8, mayúscula, minúscula, dígito y carácter especial. Nombre: 2 palabras, ≥3 letras cada una, máximo 30 caracteres, sin números ni símbolos. Activación por código de correo. 5 intentos fallidos en 15 minutos bloquean (hoy **1 minuto**; TODO volver a 30). JWT acceso 15 min, refresh 7 días. Historial de contraseñas impide reutilizar las recientes. Recuperación y reactivación no revelan si el email existe.

Evento

Fechas obligatorias; inicio hoy o futuro; inicio ≤ 2 años; fin ≥ inicio. Aforo 1–100000 y no puede superar capacidad del sitio. Único (nombre + fecha inicio + sitio). Categoría activa. Media 3–9 para enviar a

revisión / publicar. Al menos una sección con sillas. Completar solo el día **después** de `finishDate`. Cancelar evento publicado no revierte tickets automáticamente (la UI lo avisa: reembolso aparte). Edición de layout y media restringida según estado (no en revisión). Regenerar sillas choca si hay reservas/ventas (`SEAT_REGENERATION_CONFLICT`).

Sillas / stock

Reserva temporal 10 min; luego vuelve a AVAILABLE. La orden exige reserva vigente del mismo usuario. Precio = suma de precios de sección. Al cancelar orden (DRAFT o PAYMENT_PENDING) sillas a AVAILABLE. Pagado: sillas SOLD.

Estados de evento

DRAFT, PENDING_REVIEW, PUBLISHED, REJECTED, CANCELLED, COMPLETED.

Organizador: DRAFT→PENDING_REVIEW; PENDING_REVIEW→DRAFT; REJECTED→DRAFT; PUBLISHED→CANCELLED o COMPLETED.

Admin: PENDING_REVIEW→PUBLISHED o REJECTED (motivo obligatorio); PUBLISHED→CANCELLED o COMPLETED. No se publica desde DRAFT.

Estados de orden

DRAFT → PAYMENT_PENDING o CANCELLED. PAYMENT_PENDING → PAID o CANCELLED. PAID → USED o REFUNDED. CANCELLED, REFUNDED, USED terminales. Transición ilegal: 409.

Estados de ticket

PENDING, PROCESSING, PAID, REFUNDED, USED, CANCELLED. Check-in: status PAID, no expirado, sin check-in previo → USED.

Estados de pago

PENDING → APPROVED, REJECTED o ERROR. APPROVED → REFUNDED. El resto terminales. Reembolso: REQUESTED, APPROVED, PROCESSED, FAILED. Wallet-only se cierra en local; con pasarela espera webhook `payment.refunded`.

Wallet

CREDIT / REVERSAL suman; PAYMENT resta; ADJUSTMENT ±. Pago: sin wallet, mixto (reserva + pasarela) o 100% wallet. Si falla la pasarela, rollback de la reserva.

Pagos

Moneda COP. `applyWalletCredit` default false.

5. Roles y permisos

Roles en JWT: **USER**, **ORGANIZER**, **ADMIN**. Anónimo en web: catálogo, detalle, categorías, layout/sillas de consulta, temas e idiomas activos.

Capacidad	Anónimo	USER	ORGANIZER	ADMIN
Ver eventos públicos	Sí	Sí	Sí	Sí
Comprar / reservar / checkout / wallet (consulta)	No	Sí	Sí	Sí
Pedir ser organizador	No	Sí	No (ya lo es)	—
Crear/editar/borrar sus eventos, layout, media, secciones, sillas	No	No	Sí (dueño)	Layout/media/secciones/sillas sí; admin cambia estado de cualquier evento
Enviar a revisión / cancelar publicado	No	No	Sí (suyo)	Publicar/rechazar/cancelar cualquiera

Capacidad	Anónimo	USER	ORGANIZER	ADMIN
Categorías CRUD	No	No	No	Sí
Crear sitios	No	No	Sí	Sí
Aprobar organizadores	No	No	No	Sí
Reembolsar	No	No	Sí (endpoint)	Sí
Acreditar/ajustar wallet	No	No	No	Sí
Idiomas y jobs de traducción	Consulta	Consulta	Consulta	CRUD / request
Temas propios	No	Sí	Sí	Sí

Web protege rutas con `ProtectedRoute` y `localStorage.role`. Móvil: tabs Inicio/Eventos/Perfil; admin de peticiones en stack.

No existe rol STAFF. Check-in no está acotado a organizador en seguridad.

6. Flujos de punta a punta

Registro / login

Registro Gmail → correo de activación (o GET de dev) → `/activate` o pantalla Verification → login → JWT. OAuth Google/Facebook: web 302 a `/login?oauth=true&token=...`; móvil JSON. Si el nombre OAuth no cumple reglas → `needsOnboarding` → `/complete-profile`. 5 fallos → bloqueo corto. Cuenta INACTIVE → reactivar. Logout revoca sesión; otras pestañas escuchan `storage` y van a login. Access 401 → refresh; refresh fallido → logout.

Crear evento

USER pide organizador (PDF) → ADMIN aprueba. ORGANIZER: wizard `/events/create` (datos, sitio, media) → DRAFT → editor de layout (secciones, bloques, sillas, precios) → 3–9 fotos → PENDING_REVIEW. ADMIN en `/admin-events` publica o rechaza. Publicado aparece en `/events` y móvil. Organizador puede retirar a DRAFT, cancelar publicado o completar al día siguiente.

Comprar boleta y pagar

Evento PUBLISHED → elegir sillas → RESERVED 10 min (WS) → POST orden (DRAFT, tickets PENDING, QR generado) → POST pago. Sin wallet: pasarela falsa, orden PAYMENT_PENDING. Con saldo parcial: reserva wallet + pasarela. Saldo total: pago APPROVED inmediato. Webhook HMAC aprueba/rechaza: sillas SOLD, tickets PAID, o se libera stock. UI: pending hasta confirmed.

Validación de ingreso

Ticket PAID + QR en dominio. POST check-in con `validatedBy`. No hay lector en clientes: hay que probar por API.

Cancelación / reembolso

Cancelar orden solo DRAFT/PAYMENT_PENDING: tickets CANCELLED, sillas AVAILABLE. Reembolso de pago APPROVED (ADMIN/ORGANIZER + motivo). Wallet-only: cierre local (pago/orden/tickets REFUNDED, crédito de vuelta). Pasarela: webhook. Cancelar el **evento** no dispara reembolso automático.

7. Integraciones externas

Sistema	Uso real en código
PostgreSQL 17	Persistencia; Liquibase; `ddl-auto: validate`
MinIO (API S3)	Imágenes, documentos de petición, avatares; URLs públicas/presignadas

Sistema	Uso real en código
SMTP Gmail	Activación, recuperación, reactivación, notificaciones email
JWT + refresh	Emisión propia; BCrypt factor 12
OAuth2	Google y Facebook (authorization code); GSI ID token
Pasarela	`fake-payment-gateway` (puerto 4001); webhook HMAC. README menciona Wompi; no hay cliente Wompi activo
LibreTranslate	Traducciones asíncronas (reintentos 3, timeout 10 min)
Moderación	Servicio NSFW local + Sightengine (gore/nudity/weapon)
Redis / Jenkins / Sonar / K8s	Pipeline DevOps, no lógica de negocio
Correo + in-app + WebSocket	Notificaciones; push nativo no aparece implementado como FCM/APNs
OpenStreetMap / geocoding	Cliente web para mapa de sitios

Swagger: `/swagger-ui`, `/api-docs`. Actuator público.

8. Errores y casos borde ya contemplados

- Enumeración de usuarios evitada en recover y reactivate-request.
- Open redirect OAuth: whitelist de `frontend_redirect_uri`.
- Refresh concurrente en web (una sola promesa).
- Logout entre pestañas.
- Reserva de silla caducada al crear orden.
- Máquina de estados de orden/pago (409).
- Rollback transaccional si falla la pasarela con crédito de wallet.
- Reembolso wallet-only vs webhook.
- Lockout con desbloqueo automático al vencer.
- Bio con lista de palabras prohibidas y normalización leetspeak.
- Imágenes rechazadas por NSFW/Sightengine (422).
- Evento con fecha vencida no se puede enviar a revisión ni publicar.
- Admin no publica desde DRAFT.
- Completar evento demasiado pronto.
- Rating único; organizador sin nota; rate limit 10 s.
- Código de recuperación de un solo uso.
- Wallet no queda negativa.
- Constraint único de evento y check de fechas en BD.

Huecos de borde a tener en cuenta al diseñar pruebas negativas: GET tickets por `userId` ajeno; cancelar orden sin comprobar dueño; check-in sin rol staff; storage en `permitAll`; endpoint `/dev/activate-code`; lockout de 1 minuto.

9. Pendiente o incompleto (impacto en el plan de pruebas)

- README: Wompi, FR, microservicios/API Gateway vs código: pasarela fake, idiomas ES/EN/PT/JA, monolito.
- Frontend “se integrará progresivamente”: web y móvil existen, pero no cubren todo el backend.
- Sin UI de validación QR / check-in.
- Móvil sin crear evento, layout, wallet, notificaciones in-app, reembolso.
- Web sin ruta propia de forgot password; términos aún dicen Wompi.
- Traducción: jobs pending global vacío; TODO de más entidades (sites).

- Lockout 1 min y cleanup de seguridad cada intervalo corto (TODO producción).
- Endpoint de código de activación de desarrollo.
- Storage público a nivel de filtro.
- Push nativo (RF16) no evidenciado.
- Rol staff y lector QR (RF11/RF24) solo en API.
- Comentario en `RefundStatus`: “fase futura”; el flujo de reembolso **sí está** implementado.
- Pagos create/get no usan `ApiResponse` (el cliente debe tratar el cuerpo distinto).
- `httpClient` de refresh aún no está en todos los servicios web.

Con esto se pueden armar casos: felices por flujo, negativos por cada `code`, autorización por rol, concurrencia de sillas, caducidad de reserva, webhook de pago y reembolso wallet vs pasarela. Si quieres, el siguiente paso puede ser una matriz de casos de prueba (ID, precondición, pasos, resultado esperado) a partir de este inventario.